改变世界前先认识世界：Ontology 决策结构化实践

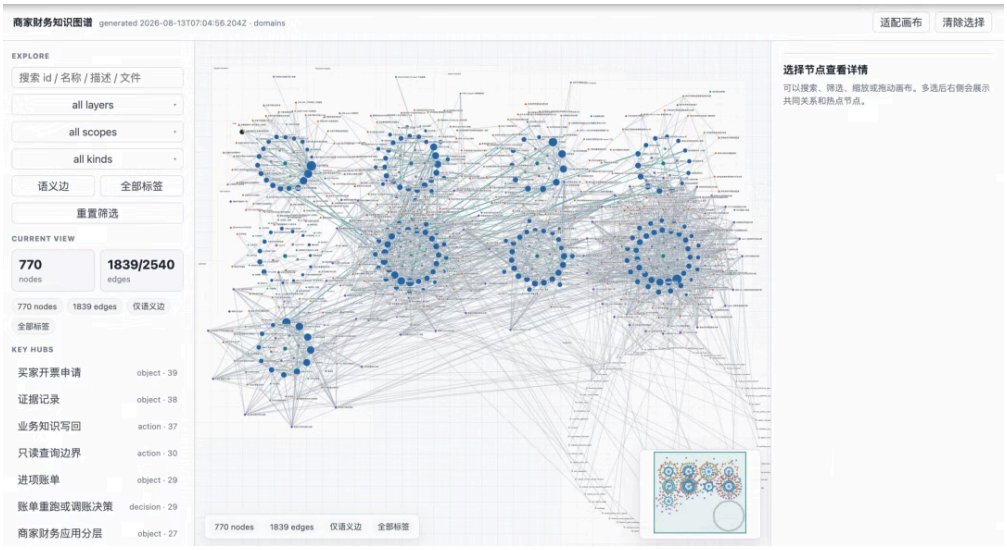
原创 供给技术团队 大淘宝技术 2026年9月27日 23:47 江苏 24人

本文介绍了商家财务团队通过 Ontology（本体）与决策结构化实践，实现 Agent 工作全托管的经验。团队摒弃了预设固定 workflow 或打包 Skill 的传统方式，将业务知识建模为包含业务、系统、交付和治理四层的 Ontology 图谱，并将专家判断固化为可执行的 decision_function。Agent 在执行时严格遵循“先取证、再决策、后行动”的闭环，根据证据动态生成工作流，完成需求开发、工单处理及数据订正等 120 多项任务。同时，团队采用碎片化知识积累模式，每天沉淀小决策产生复利。实践证明，模型和编排工具会不断迭代，但结构化的业务知识才是决定 AI Agent 能否稳定产出高质量决策的核心壁垒。

01 前言

> 3个月前我的观点：AI NATIVE 组织 / 业务自迭代，由 **AI 能否产生高质量决策 决定**。否则 Agent 集成再多的 Skill 和 CLI，也只是流程自动化，而非业务自迭代。

三个月过去，这篇文章交一张答卷。我是商家财务团队，从需求开发到工单处理、数据订正、业务运维，这三个月在 Agent Room 里创建并完成了 120+ 需求 / 工作项，几乎所有类型的工作都可以由 Agent 接手完成。文章想讲清楚三件事：我们把决策固化成了什么形态（decision_function），把知识固化成了什么形态（Ontology），以及这套知识库怎么靠"每天学一个小决策"慢慢长到能托管全部工作。



02 工作全托管长什么样

需求全托管链路，我们定义为：从需求开发到上线的整个阶段，人只需要提需求 + 批 CF/CR 单，其他操作完全不做，由 Agent 自行推进完成，并且包含安全生产验证、正式发布等节点。

在此之外，实际上我们做得更进一步，是**工作全托管**：不止于需求上线，还包括日常的数据订正、工单处理、业务接入和业务运维。只要工作上需要人投入精力的事项，都可以由 Agent 团队接手并处理完成。更关键的是，**无需预先选择工作流，所有工作流程和阶段，都是通过决策函数动态长出来的。**

工作全托管不是"把固定流程自动执行一遍"，它要求 Agent 在每一轮执行中回答这样一组问题：

问题	结构化产物
当前任务属于什么业务对象和业务线?	domain / bizLine / object
需要补哪些证据才能判断?	required_slots / evidence_matrix
哪些路径看似合理但不能走?	rejected_routes
现在可以执行什么动作?	action / workflow / skill
结果证明到了什么范围?	proof_scope
下一步交给谁?	next_owner

所以全托管的本质是每一步都能被结构化决策约束：先 Observe，再 Decide，再 Act，再 Verify，最后 Next。这五步每一步都有明确的结构化产物，可以被审计、被复现，也才能被托

管。

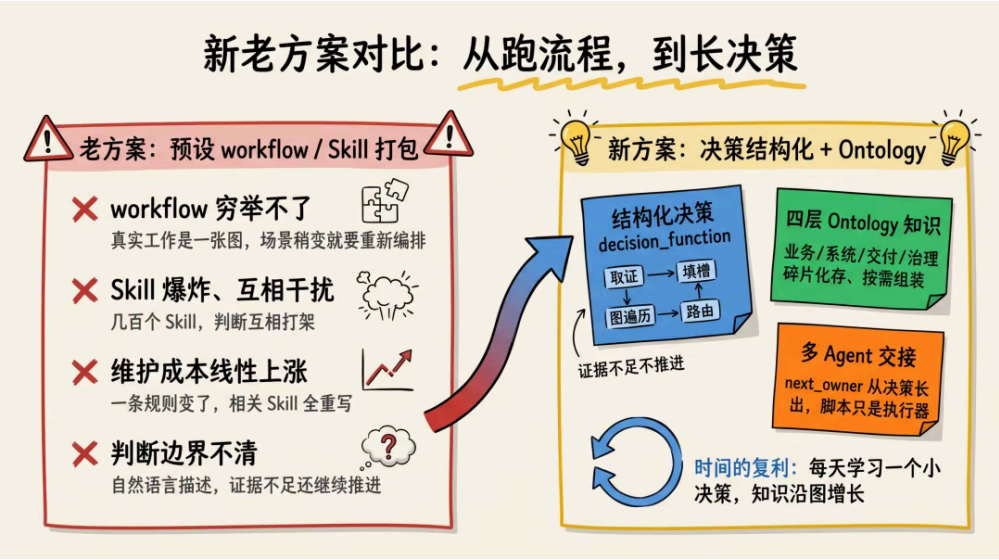
03 为什么 workflow 不能预先画好

workflow 在 Agent 时代有过几次升级迭代。最初的 workflow 依赖人手动拖拽（如 Dify），AI 几乎只起到 NL2Parameter 的作用。后来 Skill 出现后，人们发现单纯使用自然语言描述工作流，也可以让 AI 干活，而且泛化性很强，于是很快转向语言描述的 Skill。再往后，大家发现单纯自然语言描述的幻觉率较高，又开始转向内置脚本，甚至在特定场景下重新转向预设 workflow（如coco设计的需求发布流程，稳定性强、速度快）。

但从实际体验看，**Skill 定义的 workflow** 和 **预设场景 workflow** 都不是长期最优形态：

形态	优点	问题
手工拖拽 workflow	稳定、可视化、容易控制	泛化差，场景稍变就要重新编排
自然语言 Skill	灵活，覆盖长尾能力强	判断边界不清，证据不足时容易继续推进
内置脚本 Skill	可执行性强，稳定性提升	脚本解决"怎么做"，但不天然解决"该不该做"
预设场景 workflow	高频场景快、稳定	无法穷举所有真实工作场景

真实工作不是一条固定线，而是一张动态变化的图：需求可能涉及前端、后端、离线数据、配置、审批、发布、QA、运维；工单可能一开始看似是页面问题，取证后发现是账单对象、发票状态、离线回流或权限配置问题。



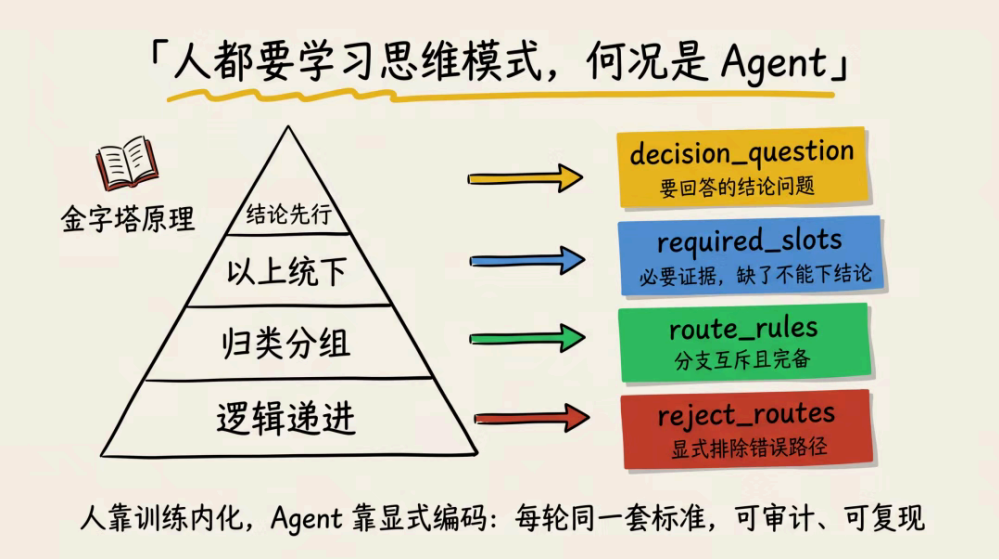
因此，我们使用的是 **决策结构化 + 决策函数** 的方式：不为每种场景提前画好 workflow，而是让 Agent 在每个阶段运行 decision_function，根据证据决定下一步。

```
传统 workflow：人先设计流程，Agent 按流程执行
决策结构化：Agent 先补证决策，再动态长出流程
```

这也是为什么"流程节点"不应该是系统的第一性原理。真正应该结构化的是决策：什么时候进入开发、什么时候补证、什么时候拒绝、什么时候交 QA、什么时候发布、什么时候必须停下来等人审批。决定了结构化决策之后，下一个问题是：结构化的决策长什么样？

04 人都要学习思维模式，何况是 Agent

曾经有本很火的书叫做《金字塔原理》，解决的是表达和思考混乱的问题。它的核心是四件事：结论先行、以上统下、归类分组、逻辑递进——先给结论，再用互斥且完备的论据支撑。这和我们写 decision_function 几乎是同构的

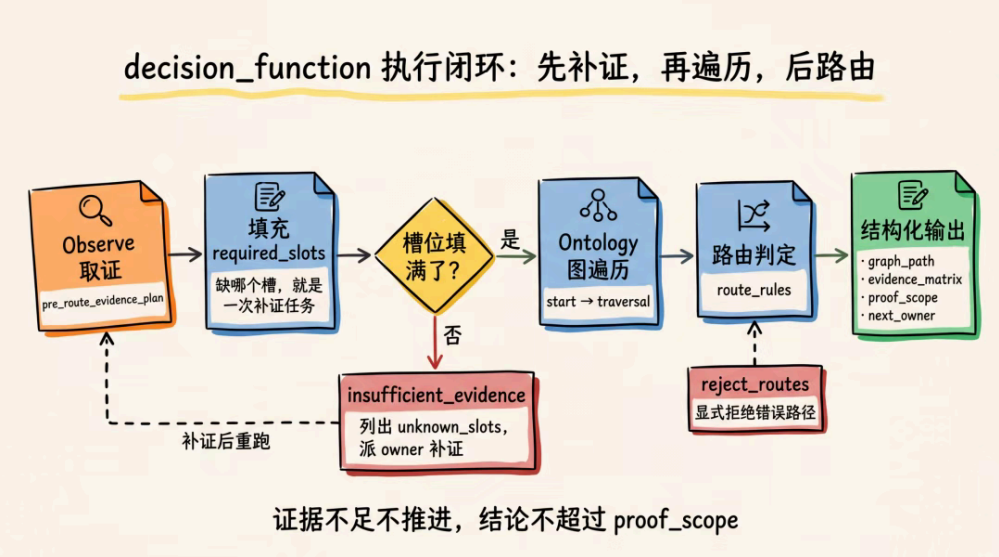


连人都需要刻意训练才能内化这套思维模式，而且训练成果没法直接复制给别人。Agent 的优势恰好在这里：思维框架一旦被写成可执行的决策程序，每一个 Agent、每一轮执行都在用同一套标准判断，不存在"今天状态不好""经验没带上"的问题。反过来说，如果不显式编码，Agent 就只能依赖模型的概率直觉做判断——同样的问题这次这么判、下次那么判，看起来每步都在推进，实际上无法审计、无法复现。

每个决策（decision）是一个结构化的判定程序，核心字段如下：

字段	作用
`decision_question`	这个决策要回答的问题
`trigger`	什么场景触发这个决策
`start`	从哪个 ontology 对象开始图遍历
`required_slots`	必须填的证据槽，缺了就证据不足，不能推进
`optional_slots`	可选槽，用于辅助判断和消歧
`pre_route_evidence_plan`	路由前先取证的步骤
`traversal`	沿 ontology 图遍历 object / relation / logic，派生结论
`route_rules`	按槽值决定路由，命中后给出原因
`reject_routes`	明确哪些路径必须拒绝，避免被关键词误导
`output`	输出 graph_path / filled_slots / unknown_slots / rejected_routes / evidence_matrix / proof_scope / next_owner

把这些字段串成一个执行闭环，整个过程没有一步依赖 Agent 的"自觉"：证据不足不推进，结论不超过 proof_scope。

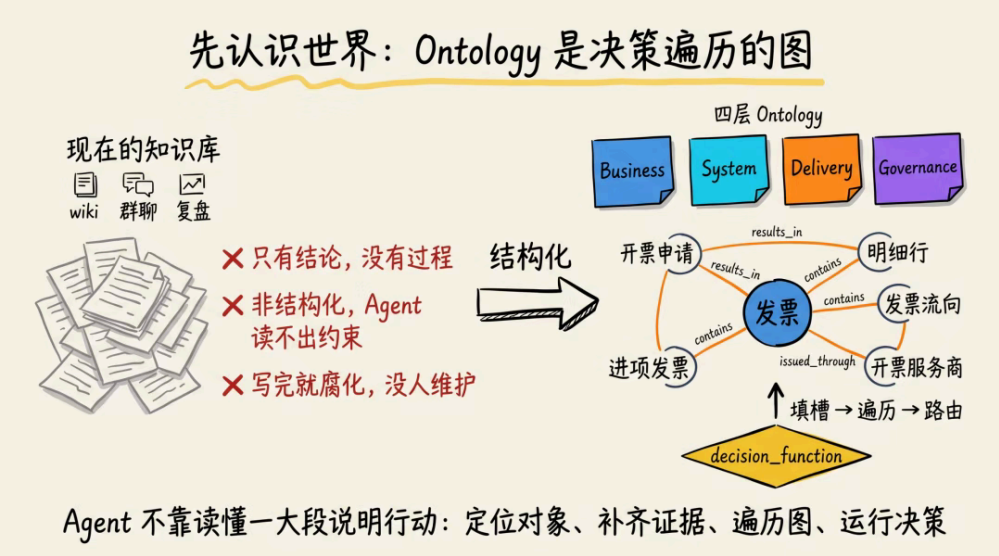


本质上，decision_function 是把"专家怎么判断"沉淀成"槽位 + 图遍历 + 路由规则"的可执行程序。Agent 必须先取证填槽再决策，不能猜。这也是为什么在 Agent Room 中要求先走 `ontology:context + decision_function`。

注意上面这套执行逻辑里，`start` 要一个起始对象，`traversal` 要关系边，`required_slots` 要证据来源——决策函数不能在空气里跑，它遍历的每张图、取的每份证据，都指向同一处：知识库。这就是故事的另一半。

05 改变世界前先认识世界：Ontology 是决策遍历的图

现在的知识库大多长什么样？一堆 wiki 页面、文档空间、群聊沉淀和事后复盘。它们有两个共同的问题。一是只有结论，没有过程：复盘里写着"这个问题最终定位是权限配置错误"，但当时查了哪些证据、排除了哪几条看起来也合理的路径，全都没有记录。下一次遇到相似问题，人或 Agent 都只能靠关键词搜索碰运气。二是非结构化：对象、属性、判断、动作混在一段段自然语言里，人读得懂，Agent 读不出确定的执行约束。而且文档写完就开始腐化，没人知道哪一段还有效。



Ontology 的由来

Ontology 在企业界有一个出名的参照：Palantir。它把 Ontology 官方定义为组织的 Operational Layer——架在数据集和模型之上的一层业务语义层，把表、字段、外键重新组织成业务人员和 AI 都能理解的对象、属性、关系和动作：客户、订单、设备、库存、发票、工单。而它的诞生来自一种必要性：前线部署工程师（FDE）在客户现场解决具体问题，但现场方案如果不抽象成通用业务本体，换个客户就带不走；Ontology 正是把离散场景沉淀为平台能力的抽象机制。

这套实践给数据分析世界带来几个明确的转向：从表驱动到对象驱动——不先问"我们有哪些表、怎么连"，先问"企业里有哪些关键业务对象"；从指标汇总到对象状态——把应收账款余额、设备故障率挂回每个客户、每台设备身上，指标变成状态信号；从单点分析到关系追踪

——订单延期要沿着供应商、物料、生产、客户的链路追到根因。到了 Agent 时代，这层语义又多了一个用途：AI 面对的不再是一堆表和字段，而是业务对象和它们的状态、权限、可执行动作，"哪个字段代表风险、哪个动作需要审批"终于有地方写了。

我们借的正是这个核心思路，但要解决的不是企业数据分析，而是 Agent 执行。所以相对 Palantir 的 Ontology，我们多做两件事：在业务对象之上又加了 Delivery 和 Governance 两层，把交付门禁和执行治理也写进同一张图；把"判断"以 decision_function 的形式挂到对象上，让 Ontology 不只给人和 AI 读，还能直接驱动 Agent 取证、决策、交接。Palantir 的 Ontology 让企业和 AI 共用一套业务语言；我们要让这套语言可执行、可审计。

Ontology 在这里不是一份自然语言知识库，而是一个可被 Agent 执行和审计的业务世界模型。它把稳定知识拆成对象、属性、关系、状态、证据、判断、动作和反馈：

```
Object / Property / Relation / State
         -> Metric / Evidence
         -> Logic / Decision
         -> Action / Workflow
         -> Audit / Feedback
```

在商家财务知识库里，Ontology 分为四层：

层级	解决的问题	示例
Business Ontology	业务世界中有哪些对象、属性、关系和判断	发票、账单、开票申请、进项票、销项票、给买家开票
System Ontology	这些业务对象落在哪些系统、服务、表、配置和链路里	在线表、离线表、服务接口、配置项、消息链路
Delivery Ontology	一个变更如何安全交付	代码、应用、本地验证、预发发布、正式发布、二方包、CR、回滚
Governance Ontology	Agent 如何读取上下文、执行决策、交接、审计和自迭代	context_pack、decision_function、proof_scope、handoff、golden case

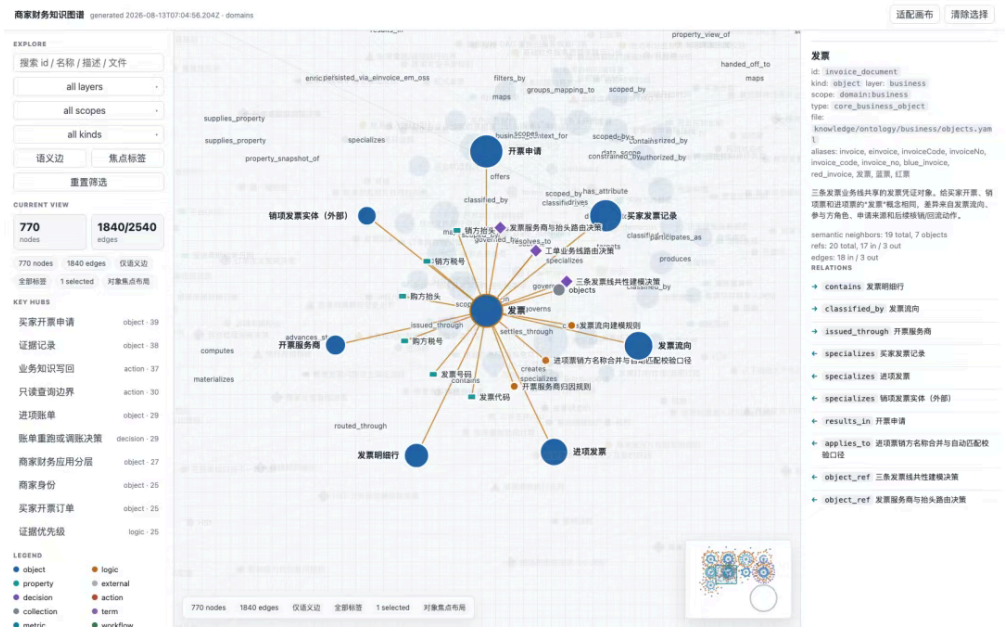
自然语言知识库适合解释背景，但不适合直接约束 Agent 行为。Ontology 的价值在于把知识拆成可检索、可组合、可验证的结构：Agent 不再靠"读懂一大段说明"行动，而是先定位对象，再补齐证据槽，沿关系图遍历，最后运行决策函数。

下面用知识图谱的真实截图，看看前三层各自长什么样、怎么发挥作用。

Business Ontology：把"发票"先变成一个稳定对象

拿"发票"举例。它在知识库里被定义成一个 core_business_object，先解决的是"同一个词、不同含义"的问题：给买家开票、销项票和进项票三条业务线共享同一个发票凭证对象，差异来自发票流向、参与方角色和申请来源。围绕这个对象，ontology 还挂了三层内容：aliases（invoice、einvoice、蓝票、红票、invoiceCode……工单或需求里提到任何一个词，都能定位到同一个对象）、key_states（created、issued、valid、invalid、blue、red、matched……发票处在什么状态，对着这组状态查，不靠猜）、relations（开票申请 results_in 发票、发票 contains 明细行、发票 has 流向、发票 issued_through 服务商——对象之间的关系显式建出来，图遍历才有边可走）。

下图是"发票"节点在知识图谱里的真实渲染：中心是 invoice_document，周围是开票申请、买家发票记录、发票明细行、进项发票、发票流向、开票服务商等语义邻居，连销方税号、购方抬头这类属性节点和"三条发票线共性建模决策"这类 decision 节点都挂在边上，右侧面板是它的 aliases、描述和关系清单。Agent 接到"买家下载不到发票"这类工单时，先定位到这个对象，再沿这些边查状态和关系，而不是靠关键词搜索碰运气。

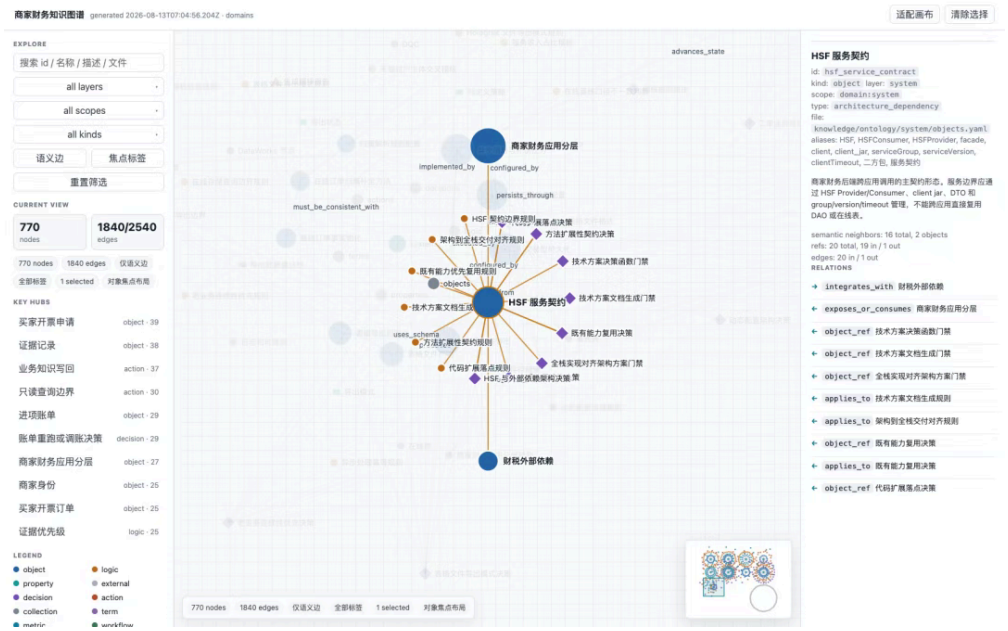


System Ontology：业务对象到底落在哪里

业务层只说"是什么"，系统层回答"在哪里"。发票对象的可信源同时登记了在线表（einvoice、einvoice_seller、einvoice_apply……）、离线表、后端代码（OutputInvoiceController、InvoiceManager……）和文档——业务对象和系统实现之间的桥，就建在对象定义里，排查时不用重新人肉梳理。

系统层里一类高频对象是离线表。离线表是在线表在 ODPS / MaxCompute 侧的镜像和加工结果，对账、报表、回流都靠它；它的来源是同步任务和 DataWorks 节点，Agent 要写离线 SQL 时，不是对着表名从零开始：先看在线表 DDL 拿字段和类型，再看后端代码（DO、mapper）确认字段语义和枚举取值；最后对照离线表的同步来源、分区和聚合粒度，才动手写。在线和离线对不上时，先列字段、时间范围、过滤条件的差异，不直接宣布哪边错。

下图是 HSF 服务契约节点：一边连着商家财务应用分层、财税外部依赖这些系统对象，一边被技术方案门禁等 decision 节点 object_ref 引用。系统层就是这样发挥作用的——它不是给人读的说明文档，而是决策函数取证时的证据源。



Delivery Ontology：把发布门禁也变成对象

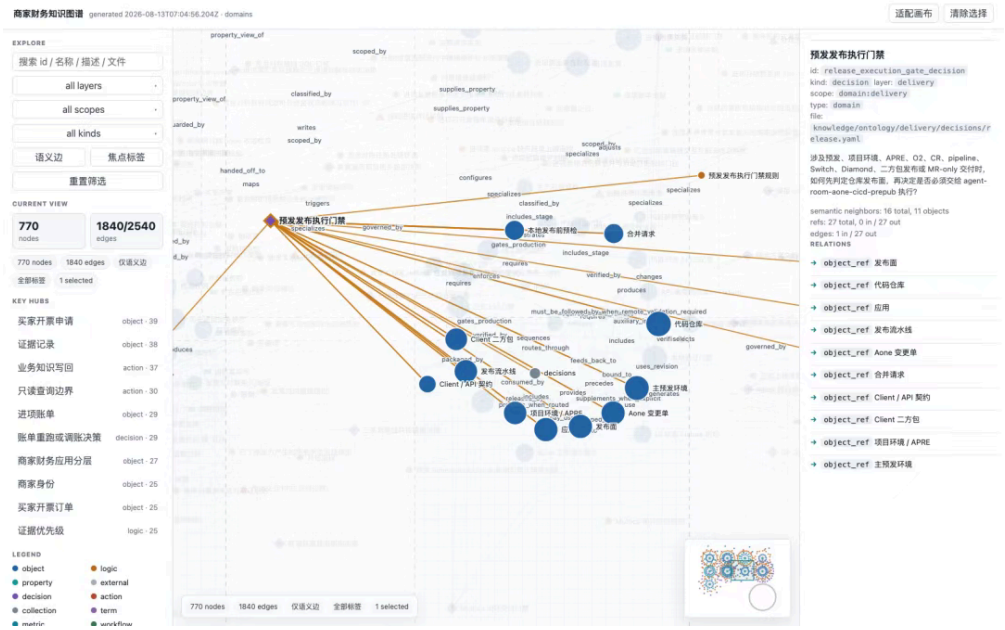
交付层的对象是代码仓库、应用、发布流水线、Aone 变更单、合并请求、代码评审门禁、项目环境、主预发环境、本地验证门禁、真实预发流程证据这一类。它们不属于任何一条业务线，但所有变更都要经过它们。最典型的是预发发布执行门禁 release_execution_gate_decision。

讲门禁之前，先让 Agent 认识两个最基础的对象：代码仓库和应用。仓库承载代码变更和 MR；应用是 Aone / O2 / VOID 平台上的可发布单元——两者是不同维度，一个仓库可能绑多个应用，也可能只是工具、文档仓库，根本没有发布流水线。所以发布判断的第一步永远是确认绑定关系（`a1 app view` 的 codeConfigs、仓库绑定信息），这一步决定了仓库的"发布面"：绑了应用且有流水线，走 CR / 流水线预发链路；前端是 O2 / VOID 应用，走 VOID publish；配置类变更，走 Switch / Diamond 推送；没应用也没流水线，才允许 MR-only 交付。我们日常的发布流程，其实就是这串对象的链：仓库 → 应用 → 发布流水线 → Aone 变更单（绑定应用、分支、revision 和工作项）→ 项目环境 / 主预发 → 门禁决策。Agent 不需要背流程，认识对象之后，流程就是对象之间边的顺序。

预发发布执行门禁的 decision_question 是：涉及预发、APRE、O2、CR、pipeline、Switch/Diamond 或二方包发布时，如何先判定仓库发布面，再决定是否必须交给发布 Agent 执行。要回答这个问题，得填二十多个 required_slots：仓库是否绑定应用、有没有发布流水线、契约是否变更、二方包是否已发远端、下游是否 fresh compile 通过、主预发是否部署到本轮 revision……每个槽都有取证来源，填不满就判 insufficient_evidence，不推进。

下图是这个决策节点在图谱里的样子：它 object_ref 出去的正是发布面、代码仓库、应用、发布流水线、变更单、合并请求、Client/API 契约、项目环境、主预发环境这些交付对象，边

上的 gates_production、requires、verified_by 就是门禁的判定关系。workflow 不是预先画好的——发布那一刻，Agent 沿这些对象走一遍，门禁路由自然长出来。



结构化、结构化、还是结构化

重复三遍结构化，是因为结构化非常重要。无论是决策的结构化，还是知识的结构化，都直接决定 Agent 能不能稳定复用经验。

自然语言知识的典型问题是：看起来信息很多，但 Agent 很难知道哪些是事实、哪些是判断、哪些是前置条件、哪些是拒绝路径、哪些只是某一次讨论的上下文。结构化知识则把这些内容拆开：

内容	自然语言知识库	Ontology 结构化知识
业务对象	混在描述里	Object
关键属性	靠文本理解	Property
对象关系	靠上下文推断	Relation
判断规则	混在经验里	Logic / Decision
证据要求	经常缺失	Evidence / required_slots
可执行动作	和判断混在一起	Action / Workflow / Skill
错误路径	往往不记录	reject_routes
证明范围	很少显式说明	proof_scope

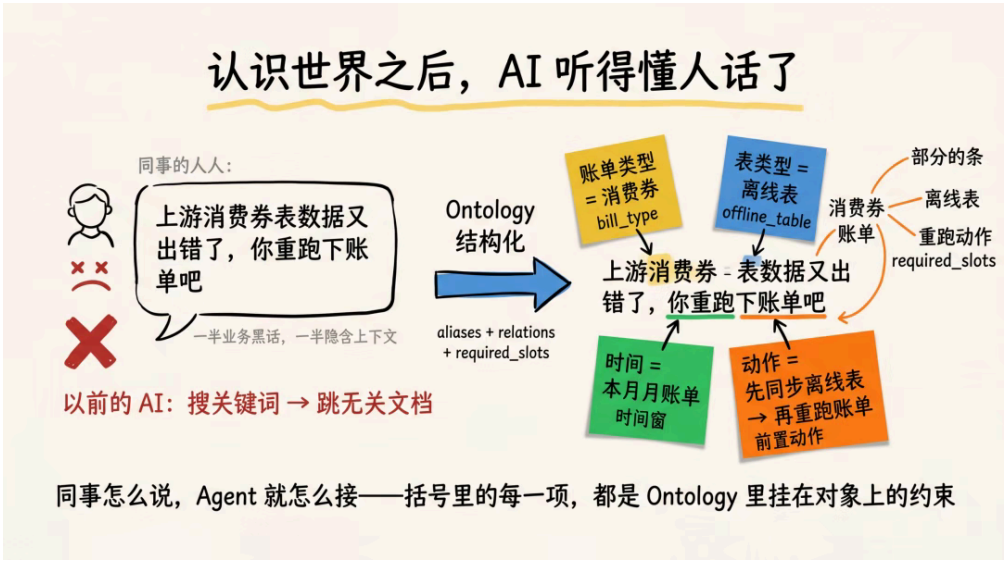
把知识做成结构化 Ontology 之后，最先跑出来的收益不是「效率」，是 AI 能听懂人话了。

日常沟通里，同事之间的对话被压缩到极致——一半是「业务黑话」，一半是「隐含上下文」。以前接工单的 AI 遇到这两种压缩，几乎都在两种失败里循环：搜关键词跳到无关文档，或者读完消息也不知道下一步该做什么。

先看第一种，「业务黑话」。项目代号（如混沌、章鱼等没有实际意义的代号），没接触过的人根本猜不到短词背后指什么，传统关键词搜索只会跳到无关文档。而在 Ontology 里，这些黑话就是对象的 `aliases`——「xxx」和「聚合结算账户平台」挂在同一个对象节点上。Agent 走一步别名解析就能定位到正确对象，接着读到属性、关系和 `source_of_truth`——工单里出现的短词就变成了对象引用。

再看第二种，「隐含上下文」，这个更棘手。月初出账时经常收到这样一句：「上游消费券表数据又出错了，你重跑下账单吧」。人一看就懂完整版：上游（账单类型为）消费券（离线）表数据又出错了，你重跑下（本月的月）账单吧，（先重跑离线表到本项目空间，然后再发起账单重跑任务）。括号里的四项——账单类型、表类型、时间范围、动作顺序——都是知识库里明确挂在对象上的约束：`bill_type` 的取值枚举、`offline_table` 的形态、月账单的时间窗口如何计算、重跑账单的前置动作是什么。Agent 拿到这句话，先在 Ontology 里定位到「消费券账单」对象，沿关系边找到它的离线表来源和账单重跑动作，再对着 `decision_function` 的 `required_slots` 检查哪几个槽被隐含了、需要补——两个动作按顺序跑起来，事就成了。

两件事拼在一起，效果就是**同事怎么说，Agent 就怎么接**。他不用改口把话补全，我们也不用给 AI 写一份翻译手册——所有「人不说也懂」的部分被结构化在 Ontology 里，成了 AI 也能看的东西。反过来看，链上任何一环缺失，AI 就会变回原来的样子：黑话没进 `aliases`，跳无关文档；对象关系没建全，查不到「消费券账单 → 离线表 → 重跑动作」的链路；决策函数缺 `required_slots`，识别不出哪些槽被隐含了。所以「AI 听得懂人话」不是模型能力问题，而是知识库结构化程度问题。



决策和 Ontology 的形态讲完了，一个自然的问题是：碎片这么好用，为什么不直接打包成 Skill？毕竟写得好的 Skill 确实能干活，前面 workflow 迭代里它也赢过预设流程。先说清楚一个好的 Skill 长什么样。一个真能用的 Skill，至少要写全五样东西：触发场景描述（什么时候用它）、完整判断链（先看什么、后看什么、怎么分支）、执行动作（跑哪些命令、调哪些接口）、注意点与兜底（哪些坑不能踩、失败了怎么办）、验证方式（什么证据证明做完了）。这五样写齐，一个 Skill 就是一份几百行的闭环小手册——它确实能干活，这也是 Skill 形态受欢迎的原因。

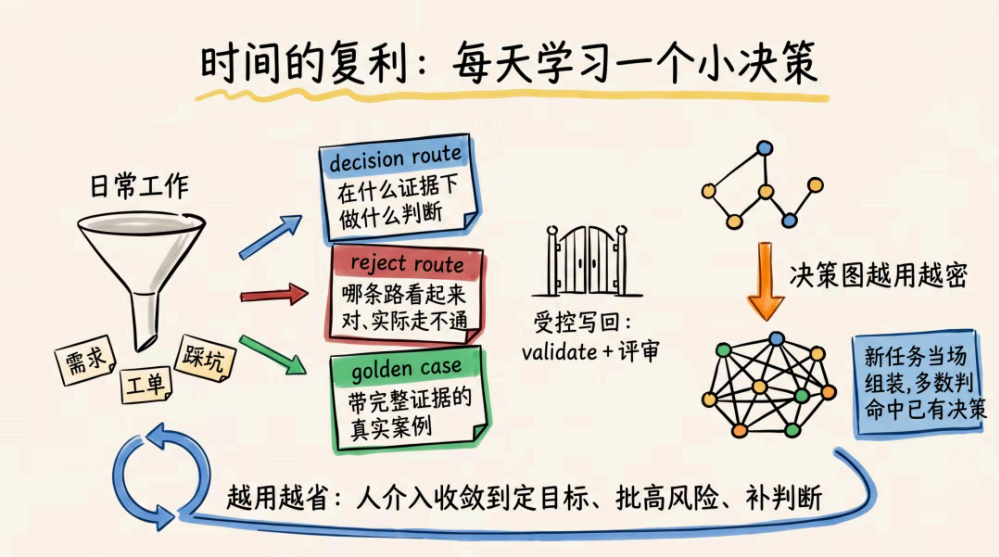
但把这份小手册拆开看，会发现它其实是 ontology 碎片沿一个场景的**固定打包**：触发场景是 decision 的 `trigger`，判断链是 `required_slots` + `route_rules` + `reject_routes`，涉及的对象和证据是 object / relation / source_of_truth，执行动作是 action / workflow，验证方式是 proof_scope + golden case。Skill 里很少有新知识，新知识都在碎片里。

问题就出在"打包"上。如果把工作中所有事项都做成独立 Skill：一是可能要产生几百个 Skill，并且 Skill 之间会互相干扰——一个任务同时命中两三个 Skill 是常态，它们的判断互相打架；二是每次编写 Skill 都要产出完整的逻辑链条和执行动作，工作量和文本量巨大，而且同一条规则变了，要把所有包含这条规则的 Skill 都改一遍，维护成本随场景数量线性上涨。还有执行期的问题，前面提过一句：自然语言描述的判断边界不清，证据不足时容易继续推进——打包的内容越多，越难用证据槽约束住它。

所以我们反过来：不预打包，碎片化存。每天遇到的需求、处理的工单、踩过的坑，解决过程都被拆成三类碎片写回知识库：

- 一条新的 decision route：在什么证据下做什么判断；
- 一条 reject route：哪条路看起来对、实际走不通；
- 一个 golden case：带完整证据的真实案例，用于回归校验后续变更会不会破坏原有判断。

执行时再由 decision_function 按任务当场组装：同一条"发布门禁"路由，可以和"发票归属"的对象组合，也可以和"数据对账"的逻辑组合。这就是相对 Skill 的核心优势——**碎片可复用、可组合、可独立演进**：新场景不用新写 Skill，用已有碎片拼；一条规则修订，只改一处，所有组合自动生效。

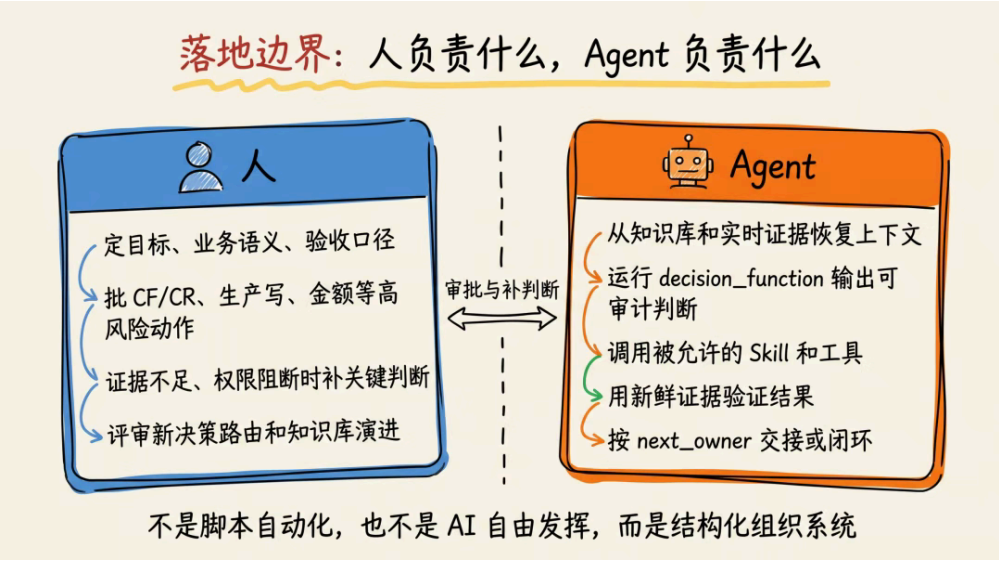


写回本身是受控的，不是随手一存：先读取已有 ontology 和 decision 约束，用工具取证，执行既有 decision_function，识别旧决策的缺口，生成 learning proposal，跑 `ontology:validate` 和案例回归，经过评审后才视为长期知识。未验证的猜测、一次性日志、临时判断，一律不进 ontology。

这种方式能产生复利，是因为决策是可组合的：新任务由已有的对象定义和已有的判断规则拼装而成，知识覆盖面沿着一张图增长，而不是一份清单。图足够密之后，新任务路径上的大多数判断点都会命中已有的 decision_function，需要人介入的部分收敛到少数几类：定目标、批高风险动作、在证据不足时补判断。目前已完成的 120+ 需求就是这样一天天攒出来的——每完成一单，除了交付本身，还会留下几条可复用的决策。

08 人和AGENT的边界

工作全托管不是取消人，而是把人的注意力从重复推进转移到目标、边界和风险确认上。



这套边界让工作全托管既不是"脚本自动化"，也不是"AI 自由发挥"。它更像一个结构化组织系统：业务知识被建模，专家判断被函数化，工具执行被门禁约束，真实结果被证据回写。

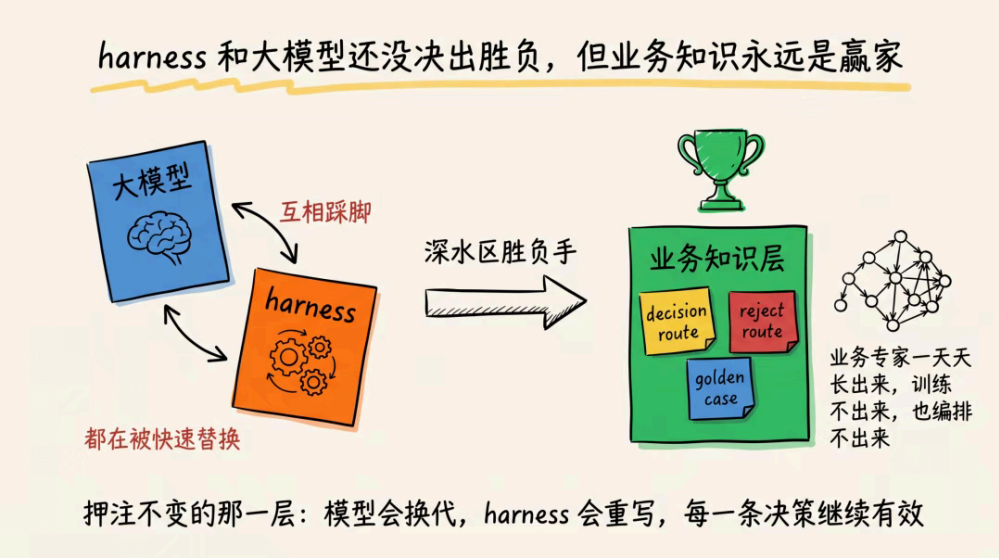
回到开头那个观点：AI NATIVE 的关键是高质量决策。决策结构化了，workflow 就不需要预先设计——它从每一轮证据充分的判断里自己长出来。这是我们从流程自动化走到工作全托管的全部路径，也是接下来继续扩场景的底座。

09
harness和大模型还没决出胜负，但业务知识永远是赢家

这一年大模型和 harness 一直在互相踩脚：以前要靠 harness 写多智能体协同、工具调用、重试逻辑，现在模型自己一把就处理掉了。同一套知识库和决策函数，我在 qoder 系列、codex、claude code 上都跑过真实需求，模型换过 opus、gpt5.5、glm5.2、qwen 3.8——平台和模型都换了一遍，输出稳定性没变。

这说明问题出在哪：模型层和编排层都在被快速替换，业务知识层没有。模型决定怎么推理，harness 决定怎么编排，但"红冲工单要取哪些证据、哪条路绝对不能走、预发门禁之后交给谁"——这些判断既不在模型权重里，也不在 harness 的编排里，它们在 ontology 和 decision_function 里。agent 越往垂直场景的深水区走，胜负越取决于这一层；而这一层只能由业务专家一天天长出来，训练不出来，也编排不出来。

所以我们不押注哪个模型、哪个 harness 会赢。模型会继续换代，harness 会继续重写，但每一条 decision route、reject route、golden case 都继续有效。



10
团队介绍

本文作者汀叶，来自淘天集团–商家与开放平台团队。团队聚焦淘天商家工具的持续升级，通过丰富官方工具与无线端能力，构建标准化接入流程与技术体系，将商家工具以统一的产品形态和交互标准交付；同时团队积极探索AI赋能商家，持续深耕AI技术在店铺经营中的应用，提升商家经营效率与平台交易增量价值；为商家提供更智能、更开放、更可靠的经营体验。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

964篇原创内容

公众号

阅读 4901

留言

写留言

还没有留言，来写首评